

MIGNet Network 特征提取报告

clean_grn_cci_mix 与 clean_grn_cci_expr_mix 的代码逻辑、数学公式与 expression
OT 链路

2026 因果涌现项目

2026-07-02

目录

1. 总结：这两个 network 方法到底是什么	3
2. 代码入口和类继承关系	4
3. 输入数据：这些特征从哪些文件来	5
4. shared genes：两个 clean 方法怎样选基因集合	6
5. GRN 权重归一化：从原始权重到 $\tilde{w}_{g_a, g_b}$	7
6. intra GRN 边：clean_grn_cci_mix 怎么计算	7
7. intra GRN 边：clean_grn_cci_expr_mix 怎么计算	8
8. inter CCI 边：两个方法完全一样，都是 CCI-only	10
9. 边表如何变成特征矩阵：native feature schema	11
9.1 intra GRN 特征列	11
9.2 inter CCI 特征列	11
10. unit 的 feature vector 具体怎么算	12
10.1 intra GRN 部分	12
10.2 inter CCI 部分	13
10.3 log1p 压缩	13
11. 一个具体例子：同一个 unit 的特征如何出现	14
12. 从 network feature 到 expression OT：这里的 expression 不是原始表达	15
13. PCA 降维和全局缩放	16
14. expression OT 的 cost matrix 怎么算	17
15. expression OT 的 Sinkhorn kernel 怎么算	18

16. lower PIJ 和 upper PIJ 是分别算的	20
17. 两个方法的差异表	20
18. 对实验解释最重要的结论	21
19. 可以在论文/报告里使用的表述	22

本报告基于 /mnt/data/code.zip.zip 的静态代码阅读。重点代码位置包括：

- mignet_ce/networks/clean_grn_cci_mix.py
- mignet_ce/networks/clean_grn_cci_expr_mix.py
- mignet_ce/graph/builder.py
- mignet_ce/features_native.py
- mignet_ce/representations/graph_features.py
- mignet_ce/pij/expr_ot.py
- mignet_ce/pij/_ot_common.py
- mignet_ce/transition/cost_components.py
- mignet_ce/transition/ot.py

本报告没有实际运行 Mouse embryo 数据，只解释代码逻辑、特征公式和 expression OT 计算链路。

1. 总结：这两个 network 方法到底是什么

当前代码里，clean_grn_cci_mix 和 clean_grn_cci_expr_mix 都属于 **network 构造器**。它们不是 PIJ 方法本身，而是为后面的 PIJ 方法准备每个层级、每个时间点、每个 unit 的特征矩阵。

设一个时间点为 t 。这里的 t 表示发育时间点，例如 $t = 11.5$ 、 $t = 12.5$ 、 $t = 13.5$ 或 $t = 14.5$ 。每一个 t 都会对应一批 lower-layer units 和一批 upper-layer units。这里的 lower-layer unit 记为 u ，其中 u 表示低层级上的一个 unit，例如一个 spot、一个 Seurat domain、一个 Louvain domain。这里的 upper-layer unit 记为 v ，其中 v 表示高层级上的一个 unit，例如更粗粒度的 domain。

这两个 network 方法都把一个 unit u 表示成一个向量：

$$\mathbf{x}_u^{(t)} \in \mathbb{R}^D.$$

在这个公式里， $\mathbf{x}_u^{(t)}$ 表示时间点 t 上 unit u 的 network feature 向量； u 是一个空间/聚类 unit； t 是发育时间点； D 是总特征维度； $\mathbb{R}^D$ 表示 D 维实数空间。这个向量 $\mathbf{x}_u^{(t)}$ 的每一维不是原始基因表达，而是由 **GRN 边**和 **CCI 边**汇总出来的网络活动特征。

两者共同点：

1. 都包含 **intra GRN 特征**，也就是同一个 unit 内部的基因调控边，例如 $g_a \rightarrow g_b$ 。这里 g_a 表示 regulator gene，也就是调控基因； g_b 表示 target gene，也就是被调控基因。
2. 都包含 **inter CCI 特征**，也就是一个 unit 向另一个 unit 的 ligand-receptor 通讯边，例如 ligand ℓ 到 receptor r 。这里 ℓ 表示配体基因或配体复合体， r 表示受体基因或受体复合体。
3. 都采用 **native_units** 特征空间。也就是说，lower 层级的每个 unit 保留为 lower 层级自己的行，upper 层级的每个 unit 保留为 upper 层级自己的行。它们不再像旧版 legacy 方法那样把 lower 特征聚合到 stable upper unit 空间。
4. inter CCI 部分都设置为 **cci_only**。这意味着 inter 细胞通讯边的影响分数只来自 COMMOT 的 CCI score，而不再把 ligand-receptor 对应的 GRN 权重乘进去。

两者核心区别:

- `clean_grn_cci_mix` 的 intra GRN 边权重只使用归一化后的 GRN 权重。换句话说, 如果某条基因调控边 $g_a \rightarrow g_b$ 存在并通过表达 mask, 那么该边对 unit u 的贡献主要是归一化 GRN 权重 $\tilde{w}_{g_a, g_b}$ 。这里 $\tilde{w}_{g_a, g_b}$ 表示调控边 $g_a \rightarrow g_b$ 的 min-max 归一化权重。
- `clean_grn_cci_expr_mix` 的 intra GRN 边权重在归一化 GRN 权重之外, 还乘上 unit 内部 regulator gene 和 target gene 的表达活性。也就是说, 它把 $\tilde{w}_{g_a, g_b}$ 进一步乘以一个表达活性项 $a_u(g_a, g_b)$ 。这里 $a_u(g_a, g_b)$ 表示在 unit u 里, 调控基因 g_a 和靶基因 g_b 的联合表达活性。

最短一句话:

`clean_grn_cci_mix`: intra 用 GRN, inter 用 CCI。

`clean_grn_cci_expr_mix`: intra 用 GRN $\times$ 表达活性, inter 仍然只用 CCI。

在上面两个公式里, `intra` 表示 unit 内部的基因调控网络边, `inter` 表示 unit 与 unit 之间的细胞通讯边, GRN 表示 gene regulatory network, CCI 表示 cell-cell interaction, 表达活性表示由该 unit 中相关基因表达量转换出来的权重项。

2. 代码入口和类继承关系

`clean_grn_cci_mix` 的构造器是:

```
class CleanGRNCCIMixBuilder(LegacyMixedGRNCCIBuilder):
    network_method = "clean_grn_cci_mix"
    inter_influence_mode = "cci_only"
    inter_grn_pair_policy = "zero_if_missing"
    include_intra_grn = True
```

这说明 `CleanGRNCCIMixBuilder` 继承自 `LegacyMixedGRNCCIBuilder`, 但它改掉了 `inter` 边的计算方式。旧版 `legacy_mixed_grn_cci` 的 `inter` 边默认是 CCI 和 GRN 的 product, 而 `clean_grn_cci_mix` 明确把 `inter` 边变成 `cci_only`。这里的 `inter_influence_mode = "cci_only"` 是关键, 因为它决定了 unit 与 unit 之间的通讯边不再混入 GRN 权重。

`clean_grn_cci_expr_mix` 的构造器是:

```
class CleanGRNCCIExpressionMixBuilder(CleanGRNCCIMixBuilder):
    network_method = "clean_grn_cci_expr_mix"
```

这说明 `CleanGRNCCIExpressionMixBuilder` 直接继承 `CleanGRNCCIMixBuilder`。因此, `clean_grn_cci_expr_mix` 完整继承了 `clean_grn_cci_mix` 的大部分流程, 包括 native feature schema、CCI-only inter 边、lower/upper 原生 unit 行空间。它只覆盖 `_grn_build_options` 和 `_network_metadata`, 把 GRN source 从 `sample` 改成 `sample_expression_weighted`。

两个方法在 registry 里的注册名是：

```
"clean_grn_cci_mix": CleanGRNCCIMixBuilder,
"clean_grn_cci_expr_mix": CleanGRNCCIEExpressionMixBuilder,
```

因此命令行参数里的 `--network-method clean_grn_cci_mix` 会调用 `CleanGRNCCIMixBuilder`，而 `--network-method clean_grn_cci_expr_mix` 会调用 `CleanGRNCCIEExpressionMixBuilder`。

3. 输入数据：这些特征从哪些文件来

对每个 organ、每个时间点、每个 layer，代码会解析出一组 `LayerPaths`。设 organ 为 o 。这里的 o 表示器官，例如 heart、brain 或 lung。设时间点为 t 。这里的 t 仍然表示发育时间点。设层级为 L 。这里的 L 表示 spot、seurat_k40、seurat_k150、louvain_k40 等层级。

每个 (o, t, L) 会读取：

1. 表达矩阵路径：

```
{data_root}/{layer}/{organ}/{sample}.h5ad
```

2. 样本级 GRN 边表路径：

```
{data_root}/grn/{layer}/{sample}/grn_edges.csv
```

3. COMMOT ligand-receptor manifest 路径：

```
{data_root}/cci/{layer}/{sample}_COMMOT_lr_pairs.tsv
```

4. COMMOT unit index 路径：

```
{data_root}/cci/{layer}/{sample}_index.tsv
```

5. 每个 ligand-receptor 对应的 CCI 稀疏矩阵目录：

```
{data_root}/cci/{layer}/{sample}_COMMOT_by_LR/
```

6. 如果 layer 不是 spot，还会读取 spot-domain mapping 路径：

```
{data_root}/{layer}/{organ}/{sample}_spot_domain_map.csv
```

表达矩阵读入后形成：

$$E^{(t,L)} \in \mathbb{R}_{\geq 0}^{N_{t,L} \times G_{t,L}}.$$

在这个公式里， $E^{(t,L)}$ 表示时间点 t 、层级 L 的表达矩阵； $N_{t,L}$ 表示这个时间点和层级上的 unit 数量； $G_{t,L}$ 表示这个时间点和层级上的基因数量； $\mathbb{R}_{\geq 0}$ 表示非负实数。矩阵元素可以写成：

$$E_{u,g}^{(t,L)}.$$

这里 $E_{u,g}^{(t,L)}$ 表示时间点 t 、层级 L 中，unit u 上基因 g 的表达量； u 是 unit； g 是基因。

GRN 边表读入后，每条边可以写作：

$$g_a \rightarrow g_b, \quad w_{g_a, g_b}.$$

这里 g_a 表示 regulator gene, g_b 表示 target gene, w_{g_a, g_b} 表示原始 GRN 权重。代码会对每个 regulator gene 只保留 top-k target, 默认 `top_k_targets_per_regulator=20`。所以 w_{g_a, g_b} 不是所有可能基因对的权重，而是经过 top-k 过滤后的 GRN 边权重。

CCI 数据由 COMMOT 生成。对每个 ligand-receptor 对 (ℓ, r) ，COMMOT 提供一个 unit-by-unit 稀疏矩阵：

$$S^{(\ell, r)} \in \mathbb{R}_{\geq 0}^{N \times N}.$$

在这个公式里， $S^{(\ell, r)}$ 表示 ligand ℓ 与 receptor r 这一对通讯通道的 CCI 分数矩阵； ℓ 表示 ligand； r 表示 receptor； N 表示当前 layer 当前时间点的 unit 数量。矩阵元素写成：

$$S_{u,v}^{(\ell, r)}.$$

这里 $S_{u,v}^{(\ell, r)}$ 表示源 unit u 向目标 unit v 通过 ligand ℓ 和 receptor r 发生通讯的 COMMOT score； u 是发送方 unit； v 是接收方 unit； ℓ 是 ligand； r 是 receptor。

4. shared genes: 两个 clean 方法怎样选基因集合

这两个 clean 方法使用 `_compute_shared_expression_genes`，只根据表达矩阵取交集。设某一个 level-pair 是 lower layer L_ℓ 到 upper layer L_u 。这里 L_ℓ 表示低层级， L_u 表示高层级。对每个时间点 t ，代码取 lower h5ad 的基因集合和 upper h5ad 的基因集合的交集：

$$\mathcal{G}_t = \mathcal{G}_{t, L_\ell}^{\text{expr}} \cap \mathcal{G}_{t, L_u}^{\text{expr}}.$$

在这个公式里， $\mathcal{G}_t$ 表示时间点 t 下 lower 与 upper 都有表达记录的共享基因集合； $\mathcal{G}_{t, L_\ell}^{\text{expr}}$ 表示时间点 t 、lower layer L_ℓ 的表达基因集合； $\mathcal{G}_{t, L_u}^{\text{expr}}$ 表示时间点 t 、upper layer L_u 的表达基因集合。

然后再跨所有时间点取交集：

$$\mathcal{G} = \bigcap_{t \in \mathcal{T}} \mathcal{G}_t.$$

在这个公式里， $\mathcal{G}$ 表示最终用于构建 clean network 的共享表达基因集合； $\mathcal{T}$ 表示所有运行

时间点的集合，例如 $\mathcal{T} = \{11.5, 12.5, 13.5, 14.5\}$; $t \in \mathcal{T}$ 表示遍历每一个时间点。

需要注意：旧版 legacy 方法的 `_compute_shared_genes` 同时要求基因出现在表达和 GRN 里，而这两个 clean 方法只要求基因出现在表达矩阵里。后续 `build_layer_graph` 会把 GRN 边限制到 $\mathcal{G}$ 内部，也就是只保留 regulator 和 target 都属于 $\mathcal{G}$ 的 GRN 边。

这个设计的含义是：clean 方法先保证 lower/upper/time 之间表达空间可比，再让 GRN 边表自己在这个表达基因空间里筛选。如果某条 GRN 边 $g_a \rightarrow g_b$ 的 g_a 或 g_b 不在 $\mathcal{G}$ ，这条边不会进入 intra GRN 特征。

5. GRN 权重归一化：从原始权重到 $\tilde{w}_{g_a, g_b}$

GRN 边表里的原始权重记为：

$$w_{g_a, g_b}.$$

这里 w_{g_a, g_b} 表示 regulator gene g_a 到 target gene g_b 的原始 GRN 权重。代码把所有保留边的原始权重做 min-max normalization，得到：

$$\tilde{w}_{g_a, g_b} = \epsilon_w + (1 - \epsilon_w) \cdot \frac{w_{g_a, g_b} - w_{\min}}{w_{\max} - w_{\min}}.$$

在这个公式里， $\tilde{w}_{g_a, g_b}$ 表示归一化后的 GRN 权重； w_{g_a, g_b} 表示原始 GRN 权重； $w_{\min}$ 表示当前边表中最小的原始 GRN 权重； $w_{\max}$ 表示当前边表中最大的原始 GRN 权重； ϵ_w 是归一化下界，代码默认是 10^{-6} 。

如果 $w_{\max} \leq w_{\min}$ ，也就是说所有 GRN 权重相等或无法形成有效范围，代码会把所有 $\tilde{w}_{g_a, g_b}$ 设为 1。这里的 $\tilde{w}_{g_a, g_b} = 1$ 表示所有保留的 GRN 边在归一化意义上强度相同。

6. intra GRN 边：clean_grn_cci_mix 怎么计算

对一个 unit u ，对一条 GRN 边 $g_a \rightarrow g_b$ ，代码先判断两个基因是否在 unit u 中 active。active mask 的定义是：

$$M_{u, g} = \mathbf{1}\{E_{u, g} > \tau\}.$$

在这个公式里， $M_{u, g}$ 表示 unit u 中基因 g 是否 active 的二值指示变量； $E_{u, g}$ 表示 unit u 中基因 g 的表达量； τ 表示表达阈值，也就是 `cfg.expr_threshold`，默认是 0； $\mathbf{1}\{\cdot\}$ 表示指示函数，如果条件为真则等于 1，否则等于 0。

如果 `native_intra_use_expression_mask=True`，默认就是 True，那么只有当 regulator gene g_a 和 target gene g_b 都 active 时，这条 intra GRN 边才进入 unit u 的特征：

$$M_{u,g_a} = 1 \quad \text{且} \quad M_{u,g_b} = 1.$$

在这个公式里, M_{u,g_a} 表示 unit u 中 regulator gene g_a 是否 active; M_{u,g_b} 表示 unit u 中 target gene g_b 是否 active。

对于 `clean_grn_cci_mix`, `grn_source="sample"`, `expression_weight_mode="none"`。因此表达活性权重恒等于 1:

$$a_u(g_a, g_b) = 1.$$

在这个公式里, $a_u(g_a, g_b)$ 表示 unit u 内 regulator gene g_a 和 target gene g_b 的表达活性权重。对于 `clean_grn_cci_mix`, 这个 $a_u(g_a, g_b)$ 固定为 1, 不会根据表达量连续变化。

所以 `clean_grn_cci_mix` 中 unit u 的 intra GRN 边影响分数是:

$$I_{u,g_a,g_b}^{\text{intra}} = \tilde{w}_{g_a,g_b} \cdot a_u(g_a, g_b) = \tilde{w}_{g_a,g_b}.$$

在这个公式里, $I_{u,g_a,g_b}^{\text{intra}}$ 表示 unit u 内部从基因 g_a 到基因 g_b 的 intra GRN influence score; u 是 unit; g_a 是 regulator gene; g_b 是 target gene; $\tilde{w}_{g_a,g_b}$ 是归一化 GRN 权重; $a_u(g_a, g_b)$ 是表达活性权重, 但在 `clean_grn_cci_mix` 中等于 1。

如果 active mask 没有通过, 则这条边不产生记录, 也就是等价于:

$$I_{u,g_a,g_b}^{\text{intra}} = 0.$$

这里 $I_{u,g_a,g_b}^{\text{intra}} = 0$ 表示 unit u 在这条 $g_a \rightarrow g_b$ 调控边上没有贡献, 原因可能是 g_a 不 active、 g_b 不 active、或者这条边没有出现在筛选后的 GRN 表里。

7. intra GRN 边: `clean_grn_cci_expr_mix` 怎么计算

`clean_grn_cci_expr_mix` 的关键变化是:

```
grn_source = "sample_expression_weighted"
expression_weight_mode = cfg.grn_expression_weight_mode
```

默认配置可以写成:

```
cfg.grn_expression_weight_mode = "geometric_mean"
cfg.grn_expression_transform = "loglp_minmax"
```

因此它会先把表达矩阵转成表达活性矩阵。

原始表达量还是:

$$E_{u,g}.$$

这里 $E_{u,g}$ 表示 unit u 中基因 g 的表达量。若使用 `log1p_minmax`，代码先做：

$$H_{u,g} = \log(1 + \max(E_{u,g}, 0)).$$

在这个公式里， $H_{u,g}$ 表示 `log1p` 变换后的表达量； $E_{u,g}$ 表示原始表达量； u 表示 unit； g 表示基因。代码会把负值截断为 0，因此这里使用 $\max(E_{u,g}, 0)$ 。

然后对每个基因 g ，在当前表达矩阵所有 unit 上做 min-max scaling：

$$A_{u,g} = \frac{H_{u,g} - H_{\min,g}}{H_{\max,g} - H_{\min,g}}.$$

在这个公式里， $A_{u,g}$ 表示 unit u 中基因 g 的转换后表达活性； $H_{u,g}$ 表示 `log1p` 后的表达量； $H_{\min,g}$ 表示基因 g 在当前 layer 当前时间点所有 unit 中的最小 H 值； $H_{\max,g}$ 表示基因 g 在当前 layer 当前时间点所有 unit 中的最大 H 值。

如果某个基因 g 的 $H_{\max,g} = H_{\min,g}$ ，代码会特殊处理。如果这个基因在所有 unit 中是常数且为正，则 $A_{u,g} = 1$ ；否则该基因的活性会变成 0。这里 $A_{u,g} = 1$ 表示这个基因虽然没有跨 unit 差异，但它在所有 unit 中都有正表达。

在默认 `geometric_mean` 模式下，GRN 边 $g_a \rightarrow g_b$ 在 unit u 上的表达活性是：

$$a_u(g_a, g_b) = \sqrt{\max(0, A_{u,g_a}) \cdot \max(0, A_{u,g_b})}.$$

在这个公式里， $a_u(g_a, g_b)$ 表示 unit u 中调控边 $g_a \rightarrow g_b$ 的表达活性； A_{u,g_a} 表示 unit u 中 regulator gene g_a 的转换后表达活性； A_{u,g_b} 表示 unit u 中 target gene g_b 的转换后表达活性； $\sqrt{\cdot}$ 表示几何平均的平方根形式。

如果配置为 `product`，则：

$$a_u(g_a, g_b) = \max(0, A_{u,g_a}) \cdot \max(0, A_{u,g_b}).$$

这里 $a_u(g_a, g_b)$ 仍然表示表达活性，只是它用乘积而不是几何平均。

如果配置为 `min`，则：

$$a_u(g_a, g_b) = \min\{\max(0, A_{u,g_a}), \max(0, A_{u,g_b})\}.$$

这里 $a_u(g_a, g_b)$ 表示两端基因表达活性中较弱的一端，因为一条调控边是否活跃通常需要 regulator 和 target 两侧都具备表达基础。

代码还允许设置 `floor`：

$$a_u(g_a, g_b) \leftarrow \max(\rho, a_u(g_a, g_b)).$$

在这个公式里， ρ 表示 `cfg.grn_expression_weight_floor`，默认是 0； $a_u(g_a, g_b)$ 表示表达活性权重。这个 `floor` 的含义是：如果一条边已经通过 active mask，但表达活性太小，可以给它一个最低活动权重。

因此, `clean_grn_cci_expr_mix` 中 unit u 的 intra GRN influence score 是:

$$I_{u,g_a,g_b}^{\text{intra,expr}} = \tilde{w}_{g_a,g_b} \cdot a_u(g_a, g_b).$$

在这个公式里, $I_{u,g_a,g_b}^{\text{intra,expr}}$ 表示 `clean_grn_cci_expr_mix` 中 unit u 对 GRN 边 $g_a \rightarrow g_b$ 的表达加权 influence score; $\tilde{w}_{g_a,g_b}$ 表示归一化 GRN 权重; $a_u(g_a, g_b)$ 表示 unit u 中该 regulator-target 对的表达活性。

这说明 `clean_grn_cci_expr_mix` 的本质是: 同样一条 GRN 边 $g_a \rightarrow g_b$, 如果在 unit u_1 中 g_a 和 g_b 都高表达, 那么 $I_{u_1,g_a,g_b}^{\text{intra,expr}}$ 会比较大; 如果在 unit u_2 中 g_a 或 g_b 表达弱, 那么 $I_{u_2,g_a,g_b}^{\text{intra,expr}}$ 会比较小。

8. inter CCI 边: 两个方法完全一样, 都是 CCI-only

对每个 ligand-receptor 对 (ℓ, r) , COMMOT 给出源 unit u 到目标 unit v 的分数:

$$S_{u,v}^{(\ell,r)}.$$

这里 $S_{u,v}^{(\ell,r)}$ 表示源 unit u 到目标 unit v 的 COMMOT raw CCI score; ℓ 是 ligand; r 是 receptor; u 是发送方 unit; v 是接收方 unit。

代码会先扫描所有 ligand-receptor 矩阵中通过 `cci_min` 阈值的非零 score, 并得到全局最小值和最大值。然后对 raw CCI score 做归一化:

$$\tilde{s}_{u,v}^{(\ell,r)} = \epsilon_s + (1 - \epsilon_s) \cdot \frac{S_{u,v}^{(\ell,r)} - S_{\min}}{S_{\max} - S_{\min}}.$$

在这个公式里, $\tilde{s}_{u,v}^{(\ell,r)}$ 表示归一化后的 CCI score; $S_{u,v}^{(\ell,r)}$ 表示 raw CCI score; $S_{\min}$ 表示当前 layer/time 下所有保留 CCI score 的最小值; $S_{\max}$ 表示当前 layer/time 下所有保留 CCI score 的最大值; ϵ_s 是归一化下界, 代码中也是 10^{-6} 。

因为 `clean_grn_cci_mix` 和 `clean_grn_cci_expr_mix` 都设置了:

```
inter_influence_mode = "cci_only"
```

所以 inter CCI influence score 是:

$$I_{u,v,\ell,r}^{\text{inter}} = \tilde{s}_{u,v}^{(\ell,r)}.$$

在这个公式里, $I_{u,v,\ell,r}^{\text{inter}}$ 表示源 unit u 到目标 unit v 通过 ligand ℓ 和 receptor r 的 inter CCI influence score; $\tilde{s}_{u,v}^{(\ell,r)}$ 表示归一化 CCI score。

这也是 “clean” 的核心含义之一: 旧的 legacy mixed 网络中, inter 边可能是:

$$I_{u,v,\ell,r}^{\text{inter,legacy}} = \tilde{s}_{u,v}^{(\ell,r)} \cdot \tilde{w}_{\ell,r}.$$

在这个 legacy 公式里, $I_{u,v,\ell,r}^{\text{inter,legacy}}$ 表示旧版混合网络的 inter influence score; $\tilde{s}_{u,v}^{(\ell,r)}$ 表示归一化 CCI score; $\tilde{w}_{\ell,r}$ 表示 ligand ℓ 到 receptor r 这一基因对在 GRN 里的归一化权重。但是 clean 方法不再使用 $\tilde{w}_{\ell,r}$ 乘 inter CCI, 直接用 $\tilde{s}_{u,v}^{(\ell,r)}$ 。

还需要注意默认配置:

```
native_cci_inter_use_expression_mask = False
cci_inter_require_coords = False
```

因此默认情况下, inter CCI 边不要求 ligand/receptor 在表达矩阵中通过 active mask, 也不要求坐标存在。COMMOT 的 CCI score 本身成为 inter 边的主体。如果你把 `native_cci_inter_use_expression_mask=True`, 代码才会要求 ligand 和 receptor 与表达矩阵、active mask 更严格地对齐。

9. 边表如何变成特征矩阵: native feature schema

构建完 lower graphs 和 upper graphs 后, 代码调用:

```
schema = build_native_feature_schema([*lower_graphs, *upper_graphs])
```

这个 schema 会扫描所有时间点、lower/upper 所有 graph 中出现过的 intra GRN gene pair 和 inter CCI ligand-receptor pair, 然后建立统一特征列。

9.1 intra GRN 特征列

每一条不同的 regulator-target gene pair (g_a, g_b) 会形成一列:

```
intra_grn:g_a->g_b
```

数学上可以把这类列集合记为:

$$\mathcal{F}_{\text{GRN}} = \{(g_a, g_b) : g_a \rightarrow g_b \text{ 在至少一个 graph 中出现}\}.$$

在这个公式里, $\mathcal{F}_{\text{GRN}}$ 表示所有 intra GRN 特征列的集合; (g_a, g_b) 表示一条 regulator-target gene pair; g_a 表示 regulator gene; g_b 表示 target gene。

9.2 inter CCI 特征列

每一个不同的 ligand-receptor channel 会形成一列:

```
inter_cci:ligand->receptor[lr_key]
```

数学上可以把这类列集合记为:

$$\mathcal{F}_{\text{CCI}} = \{(k, \ell, r) : (k, \ell, r) \text{ 在至少一个 graph 中出现}\}.$$

在这个公式里, $\mathcal{F}_{\text{CCI}}$ 表示所有 inter CCI 特征列的集合; k 表示 COMMOT manifest 中的 `lr_key`; ℓ 表示 ligand; r 表示 receptor。

最终总特征集合是:

$$\mathcal{F} = \mathcal{F}_{\text{GRN}} \cup \mathcal{F}_{\text{CCI}}.$$

在这个公式里, $\mathcal{F}$ 表示最终 native network feature 的所有列; $\mathcal{F}_{\text{GRN}}$ 表示 intra GRN 特征列; $\mathcal{F}_{\text{CCI}}$ 表示 inter CCI 特征列。

总维度是:

$$D = |\mathcal{F}| = |\mathcal{F}_{\text{GRN}}| + |\mathcal{F}_{\text{CCI}}|.$$

在这个公式里, D 表示 feature vector 的维度; $|\mathcal{F}_{\text{GRN}}|$ 表示 GRN 特征列数量; $|\mathcal{F}_{\text{CCI}}|$ 表示 CCI 特征列数量。

10. unit 的 feature vector 具体怎么算

对某个 graph, 也就是某个时间点 t 、某个 layer L 的网络, 代码构建一个矩阵:

$$X^{(t,L)} \in \mathbb{R}^{N_{t,L} \times D}.$$

在这个公式里, $X^{(t,L)}$ 表示时间点 t 、层级 L 的 network feature matrix; $N_{t,L}$ 表示该时间点该层级的 unit 数量; D 表示统一 feature schema 的维度。

矩阵的一行是:

$$\mathbf{x}_u^{(t,L)}.$$

这里 $\mathbf{x}_u^{(t,L)}$ 表示时间点 t 、层级 L 上 unit u 的 network feature vector。

10.1 intra GRN 部分

对于 intra GRN 列 (g_a, g_b) , 代码先按 `src_unit`, `src_gene`, `dst_gene` 分组, 并对 `influence_score` 求和。因此:

$$X_{u,(g_a,g_b)}^{(t,L)} = \sum_{e \in \mathcal{E}^{\text{intra}}(u, g_a, g_b)} I_e.$$

在这个公式里, $X_{u,(g_a,g_b)}^{(t,L)}$ 表示 unit u 在 GRN 特征列 (g_a, g_b) 上的数值; $\mathcal{E}^{\text{intra}}(u, g_a, g_b)$

表示所有属于 unit u 、regulator 为 g_a 、target 为 g_b 的 intra 边记录； I_e 表示边 e 的 influence score。

因为一条 unit 内的同一基因对通常只有一条边，所以直观上就是：

$$X_{u,(g_a,g_b)}^{(t,L)} \approx I_{u,g_a,g_b}^{\text{intra}}.$$

在这个公式里， $X_{u,(g_a,g_b)}^{(t,L)}$ 是特征矩阵中的数值， $I_{u,g_a,g_b}^{\text{intra}}$ 是前面定义的 intra influence score。

10.2 inter CCI 部分

对于 inter CCI 列 (k, ℓ, r) ，代码不是把目标 unit v 作为一个单独列维度，而是把同一个源 unit u 发出的同一 ligand-receptor channel 汇总到同一列。代码逻辑是：

1. 对每个 (src_unit, dst_unit, lr_key, ligand, receptor)，如果重复出现，取最大 score。
2. 然后对同一个 src_unit 和同一个 (lr_key, ligand, receptor)，跨不同 dst_unit 求和。

因此可以写成：

$$X_{u,(k,\ell,r)}^{(t,L)} = \sum_{v \neq u} I_{u,v,\ell,r,k}^{\text{inter}}.$$

在这个公式里， $X_{u,(k,\ell,r)}^{(t,L)}$ 表示 unit u 在 CCI 特征列 (k, ℓ, r) 上的数值； k 表示 lr_key； ℓ 表示 ligand； r 表示 receptor； v 表示接收方 unit； $I_{u,v,\ell,r,k}^{\text{inter}}$ 表示源 unit u 到目标 unit v 在该 ligand-receptor channel 上的 influence score。

因为 clean 方法的 inter 是 CCI-only，所以可以进一步写成：

$$X_{u,(k,\ell,r)}^{(t,L)} = \sum_{v \neq u} \tilde{s}_{u,v}^{(\ell,r)}.$$

在这个公式里， $\tilde{s}_{u,v}^{(\ell,r)}$ 表示源 unit u 到目标 unit v 的归一化 CCI score。

这句话非常关键：**clean native feature** 的 CCI 部分描述的是一个 unit 作为发送方时，通过某个 ligand-receptor channel 向外发出多少通讯强度，而不是它具体连到了哪个目标 unit。目标 unit v 在构造边时参与计算，但最终特征列没有把 v 单独保留下来。

10.3 log1p 压缩

默认 feature_log1p=True，所以矩阵最后会做：

$$X_{u,d}^{(t,L)} \leftarrow \log(1 + X_{u,d}^{(t,L)}).$$

在这个公式里， $X_{u,d}^{(t,L)}$ 表示 feature matrix 中 unit u 、第 d 个特征维度的值； d 表示特征列

索引; $\log(1+x)$ 用来压缩大值, 避免少数强边完全支配后续距离计算。

11. 一个具体例子：同一个 unit 的特征如何出现

假设在时间点 $t = 11.5$, 某个 layer 有 unit u_1, u_2, u_3 。这里 u_1, u_2, u_3 都表示当前 layer 的不同 unit。假设共享基因里有 A, B, C, L, R 。这里 A, B, C, L, R 都是基因名, 其中 L 可以作为 ligand, R 可以作为 receptor。

假设 GRN 有两条边:

$$A \rightarrow B, \quad \tilde{w}_{A,B} = 0.8.$$

这里 A 表示 regulator gene, B 表示 target gene, $\tilde{w}_{A,B} = 0.8$ 表示归一化 GRN 权重为 0.8。

$$B \rightarrow C, \quad \tilde{w}_{B,C} = 0.3.$$

这里 B 表示 regulator gene, C 表示 target gene, $\tilde{w}_{B,C} = 0.3$ 表示归一化 GRN 权重为 0.3。

假设在 unit u_1 中, A, B, C 都 active。这里 active 表示表达量超过阈值 τ 。

对于 clean_grn_cci_mix:

$$I_{u_1,A,B}^{\text{intra}} = 0.8, \quad I_{u_1,B,C}^{\text{intra}} = 0.3.$$

在这个公式里, $I_{u_1,A,B}^{\text{intra}}$ 表示 unit u_1 中边 $A \rightarrow B$ 的 intra influence score; $I_{u_1,B,C}^{\text{intra}}$ 表示 unit u_1 中边 $B \rightarrow C$ 的 intra influence score。

对于 clean_grn_cci_expr_mix, 假设表达活性是:

$$a_{u_1}(A, B) = 0.5, \quad a_{u_1}(B, C) = 0.2.$$

这里 $a_{u_1}(A, B)$ 表示 unit u_1 中边 $A \rightarrow B$ 的表达活性; $a_{u_1}(B, C)$ 表示 unit u_1 中边 $B \rightarrow C$ 的表达活性。

那么:

$$I_{u_1,A,B}^{\text{intra,expr}} = 0.8 \times 0.5 = 0.4,$$

这里 $I_{u_1,A,B}^{\text{intra,expr}}$ 表示 expression-weighted GRN influence score。

$$I_{u_1,B,C}^{\text{intra,expr}} = 0.3 \times 0.2 = 0.06.$$

这里 $I_{u_1,B,C}^{\text{intra,expr}}$ 也是 expression-weighted GRN influence score, 只不过对应的是边 $B \rightarrow$

C 。

再假设 COMMOT 给出 ligand L 、receptor R 的 CCI 分数：

$$\tilde{s}_{u_1, u_2}^{(L, R)} = 0.7, \quad \tilde{s}_{u_1, u_3}^{(L, R)} = 0.4.$$

这里 $\tilde{s}_{u_1, u_2}^{(L, R)}$ 表示 unit u_1 到 unit u_2 的归一化 CCI score; $\tilde{s}_{u_1, u_3}^{(L, R)}$ 表示 unit u_1 到 unit u_3 的归一化 CCI score。

因为 inter 是 CCI-only, 所以两个 clean 方法都会得到：

$$I_{u_1, u_2, L, R}^{\text{inter}} = 0.7, \quad I_{u_1, u_3, L, R}^{\text{inter}} = 0.4.$$

最终 unit u_1 在 `inter_cci:L->R[...]` 这一列的值是：

$$X_{u_1, (L, R)} = 0.7 + 0.4 = 1.1.$$

如果 `feature_log1p=True`, 则实际进入后续 PIJ 的 raw matrix 值是：

$$X_{u_1, (L, R)} \leftarrow \log(1 + 1.1).$$

这里 $X_{u_1, (L, R)}$ 表示 unit u_1 在 ligand-receptor channel (L, R) 上的特征值。

12. 从 network feature 到 expression OT: 这里的 expression 不是原始表达

`expr_ot.py` 的 `ExprOTPiMethod` 叫 `expr_ot`。但在这套代码里, 它使用的是下面这个函数得到的 feature:

```
build_graph_feature_result(context, ...)
```

也就是说, 当 network method 是 `clean_grn_cci_mix` 或 `clean_grn_cci_expr_mix` 时, `expr_ot` 中的 “expression cost” 实际是 **network feature vector 之间的距离**, 不是直接对原始 gene expression matrix 做 OT。

这点很重要, 可以写成:

$$\text{expr_ot}(\text{clean_grn_cci_mix}) \neq \text{raw gene expression OT}.$$

在这个公式里, 左边表示 network method 为 `clean_grn_cci_mix` 时运行 `expr_ot`; 右边表示直接用原始基因表达做 OT。两者不相等, 因为前者的输入是 GRN+CCI 形成的 network feature。

真正的 raw expression 路线是 `pure_expression_ot` 配合 `expression_only` network, 而不是这里讨论的 `expr_ot`。

13. PCA 降维和全局缩放

对于每个时间点 t , network builder 输出 lower 特征矩阵和 upper 特征矩阵:

$$X^{L,t} \in \mathbb{R}^{N_{L,t} \times D}, \quad X^{U,t} \in \mathbb{R}^{N_{U,t} \times D}.$$

在这个公式里, $X^{L,t}$ 表示 lower layer 在时间点 t 的 network feature matrix; $X^{U,t}$ 表示 upper layer 在时间点 t 的 network feature matrix; $N_{L,t}$ 表示 lower layer 在时间点 t 的 unit 数量; $N_{U,t}$ 表示 upper layer 在时间点 t 的 unit 数量; D 表示原始 native feature 维度。

`expr_ot` 会调用 `build_graph_feature_result`。如果 `cfg.pij_feature_components=30`, 默认就是 30, 那么代码会把所有时间点、lower、upper 的矩阵堆叠起来训练 PCA:

$$X^{\text{all}} = \begin{bmatrix} X^{L,t_1} \\ X^{L,t_2} \\ \vdots \\ X^{U,t_1} \\ X^{U,t_2} \\ \vdots \end{bmatrix}.$$

在这个公式里, X^{all} 表示把所有 lower 和 upper、所有时间点的 feature row 合并后的总矩阵; X^{L,t_1} 表示 lower layer 在第一个时间点的特征矩阵; X^{U,t_1} 表示 upper layer 在第一个时间点的特征矩阵。

PCA 得到 K 个主成分, 其中:

$$K = \min(30, \text{样本行数}, D).$$

在这个公式里, K 表示实际 PCA 维度; 30 是默认请求维度; D 是原始 feature 维度。

PCA 变换可以写作:

$$Z^{L,t} = (X^{L,t} - \mu) V_K.$$

在这个公式里, $Z^{L,t}$ 表示 lower layer 在时间点 t 的 PCA 后特征; $X^{L,t}$ 表示 lower layer 的原始 network feature; μ 表示 PCA 训练时的全局均值向量; V_K 表示前 K 个 PCA 主成分组成的投影矩阵; K 表示 PCA 维度。

upper layer 同理:

$$Z^{U,t} = (X^{U,t} - \mu) V_K.$$

这里 $Z^{U,t}$ 表示 upper layer 在时间点 t 的 PCA 后特征。

然后代码分别对 lower 和 upper 做 MinMaxScaler。lower 的缩放是:

$$\tilde{Z}_{i,k}^{L,t} = \frac{Z_{i,k}^{L,t} - m_k^L}{M_k^L - m_k^L}.$$

在这个公式里, $\tilde{Z}_{i,k}^{L,t}$ 表示 lower layer 中第 i 个 unit 在第 k 个 PCA 维度上的缩放后值; $Z_{i,k}^{L,t}$ 表示 PCA 后值; m_k^L 表示 lower 所有时间点中第 k 维的最小值; M_k^L 表示 lower 所有时间点中第 k 维的最大值。

upper 的缩放是:

$$\tilde{Z}_{j,k}^{U,t} = \frac{Z_{j,k}^{U,t} - m_k^U}{M_k^U - m_k^U}.$$

在这个公式里, $\tilde{Z}_{j,k}^{U,t}$ 表示 upper layer 中第 j 个 unit 在第 k 个 PCA 维度上的缩放后值; $Z_{j,k}^{U,t}$ 表示 PCA 后值; m_k^U 表示 upper 所有时间点中第 k 维的最小值; M_k^U 表示 upper 所有时间点中第 k 维的最大值。

注意: PCA 是 lower 和 upper 合起来拟合的, 但 MinMaxScaler 是 lower 和 upper 分开拟合的。也就是说, lower 的 feature scale 和 upper 的 feature scale 是分别归一化的。

14. expression OT 的 cost matrix 怎么算

对一个时间转移 $t_0 \rightarrow t_1$, `expr_ot` 分别为 lower 和 upper 构造 PIJ。以 lower 为例。

设 source features 是:

$$A = \tilde{Z}^{L,t_0} \in \mathbb{R}^{n \times K}.$$

在这个公式里, A 表示 source time t_0 的 lower feature matrix; n 表示 t_0 的 lower unit 数量; K 表示 PCA/特征维度。

设 target features 是:

$$B = \tilde{Z}^{L,t_1} \in \mathbb{R}^{m \times K}.$$

在这个公式里, B 表示 target time t_1 的 lower feature matrix; m 表示 t_1 的 lower unit 数量; K 仍然表示特征维度。

默认 `cfg.pij_cost_metric="cosine"`, 所以代码先对每一行做 L2 normalize:

$$\hat{A}_{i,:} = \frac{A_{i,:}}{\|A_{i,:}\|_2}, \quad \hat{B}_{j,:} = \frac{B_{j,:}}{\|B_{j,:}\|_2}.$$

在这个公式里, $\hat{A}_{i,:}$ 表示 source 中第 i 个 unit 的 L2 归一化特征; $A_{i,:}$ 表示 source 中第 i 个 unit 的原始缩放后特征; $\|A_{i,:}\|_2$ 表示向量 $A_{i,:}$ 的 L2 范数; $\hat{B}_{j,:}$ 表示 target 中第 j 个 unit 的 L2

归一化特征; $B_{j,:}$ 表示 target 中第 j 个 unit 的原始缩放后特征。

然后计算 cosine distance:

$$C_{ij}^{\text{raw}} = 1 - \widehat{A}_{i,:} \cdot \widehat{B}_{j,:}^{\top}.$$

在这个公式里, C_{ij}^{raw} 表示 source unit i 和 target unit j 之间的原始 cosine cost; $\widehat{A}_{i,:}$ 表示 source unit i 的归一化 feature vector; $\widehat{B}_{j,:}$ 表示 target unit j 的归一化 feature vector; 点乘 $\widehat{A}_{i,:} \cdot \widehat{B}_{j,:}^{\top}$ 表示两个 feature vector 的 cosine similarity。

代码还会把 cost matrix min-max 归一化到 $[0, 1]$:

$$C_{ij} = \frac{C_{ij}^{\text{raw}} - C_{\min}^{\text{raw}}}{C_{\max}^{\text{raw}} - C_{\min}^{\text{raw}}}.$$

在这个公式里, C_{ij} 表示归一化后的 OT cost; C_{ij}^{raw} 表示原始 cosine cost; $C_{\min}^{\text{raw}}$ 表示整个 cost matrix 的最小值; $C_{\max}^{\text{raw}}$ 表示整个 cost matrix 的最大值。

如果配置为 euclidean, 则 raw cost 是:

$$C_{ij}^{\text{raw}} = \|A_{i,:} - B_{j,:}\|_2.$$

在这个公式里, C_{ij}^{raw} 表示 source unit i 和 target unit j 之间的欧氏距离; $A_{i,:}$ 表示 source feature vector; $B_{j,:}$ 表示 target feature vector; $\|\cdot\|_2$ 表示 L2 范数。

15. expression OT 的 Sinkhorn kernel 怎么算

`expr_ot` 只有一个 cost component, 名字叫 `expression`, 权重是:

$$\lambda_{\text{expr}} = \text{cfg.pij_expr_weight}.$$

在这个公式里, λ_{expr} 表示 expression cost 的权重; 默认 $\lambda_{\text{expr}} = 1.0$ 。因为只有一个 cost component, 所以 combined cost 仍然就是归一化后的 expression cost:

$$C_{ij}^{\text{combined}} = C_{ij}.$$

在这个公式里, C_{ij}^{combined} 表示组合后的 OT cost; C_{ij} 表示 expression feature distance cost。然后代码调用 entropic OT。平衡 OT 的目标可以写作:

$$\Gamma^* = \arg \min_{\Gamma \geq 0} \sum_{i=1}^n \sum_{j=1}^m \Gamma_{ij} C_{ij} + \varepsilon \sum_{i=1}^n \sum_{j=1}^m \Gamma_{ij} (\log \Gamma_{ij} - 1),$$

并满足:

$$\Gamma \mathbf{1}_m = \mathbf{a}, \quad \Gamma^\top \mathbf{1}_n = \mathbf{b}.$$

在这些公式里, Γ^* 表示最优 transport plan; Γ_{ij} 表示 source unit i 到 target unit j 的运输质量; C_{ij} 表示 source unit i 到 target unit j 的 cost; ε 表示熵正则强度, 也就是 `cfg.pij_entropy_epsilon`, 默认 0.05; $\mathbf{1}_m$ 表示长度为 m 的全 1 向量; $\mathbf{1}_n$ 表示长度为 n 的全 1 向量; $\mathbf{a}$ 表示 source side 的质量分布; $\mathbf{b}$ 表示 target side 的质量分布。

代码默认使用均匀质量:

$$a_i = \frac{1}{n}, \quad b_j = \frac{1}{m}.$$

在这个公式里, a_i 表示第 i 个 source unit 的质量; n 表示 source unit 数量; b_j 表示第 j 个 target unit 的质量; m 表示 target unit 数量。

从 cost 到 kernel 的直觉是:

$$K_{ij} = \exp\left(-\frac{C_{ij}}{\varepsilon}\right).$$

在这个公式里, K_{ij} 表示由 cost 转成的相似度 kernel; C_{ij} 表示 cost; ε 表示熵正则强度。 C_{ij} 越小, K_{ij} 越大; C_{ij} 越大, K_{ij} 越小。

Sinkhorn 通过缩放向量 $\mathbf{u}$ 和 $\mathbf{v}$ 得到:

$$\Gamma = \text{diag}(\mathbf{u})K \text{diag}(\mathbf{v}).$$

在这个公式里, Γ 表示 transport plan; K 表示由 cost 经过指数变换得到的 kernel; $\mathbf{u}$ 表示 source 侧缩放向量; $\mathbf{v}$ 表示 target 侧缩放向量; $\text{diag}(\mathbf{u})$ 表示以 $\mathbf{u}$ 为对角线的对角矩阵; $\text{diag}(\mathbf{v})$ 表示以 $\mathbf{v}$ 为对角线的对角矩阵。

最后代码会把 transport plan 行归一化, 得到真正保存和使用的 PIJ transition kernel:

$$P_{ij} = \frac{\Gamma_{ij}}{\sum_{j'=1}^m \Gamma_{ij'}}.$$

在这个公式里, P_{ij} 表示从 source unit i 转移到 target unit j 的 PIJ 概率; Γ_{ij} 表示 transport plan 中 source unit i 到 target unit j 的运输质量; $\sum_{j'=1}^m \Gamma_{ij'}$ 表示 source unit i 这一行所有 target 的运输质量总和。

所以 P 是 row-stochastic matrix, 也就是:

$$\sum_{j=1}^m P_{ij} = 1.$$

在这个公式里, P_{ij} 表示 PIJ transition probability; 对同一个 source unit i , 所有 target unit j 的概率和为 1。

16. lower PIJ 和 upper PIJ 是分别算的

对每个时间对 (t_0, t_1) ，代码分别计算：

$$P_{t_0 \rightarrow t_1}^L \in \mathbb{R}^{N_{L,t_0} \times N_{L,t_1}},$$

$$P_{t_0 \rightarrow t_1}^U \in \mathbb{R}^{N_{U,t_0} \times N_{U,t_1}}.$$

在第一个公式里， $P_{t_0 \rightarrow t_1}^L$ 表示 lower layer 从时间点 t_0 到时间点 t_1 的 PIJ transition matrix； N_{L,t_0} 表示 lower layer 在时间点 t_0 的 unit 数量； N_{L,t_1} 表示 lower layer 在时间点 t_1 的 unit 数量。

在第二个公式里， $P_{t_0 \rightarrow t_1}^U$ 表示 upper layer 从时间点 t_0 到时间点 t_1 的 PIJ transition matrix； N_{U,t_0} 表示 upper layer 在时间点 t_0 的 unit 数量； N_{U,t_1} 表示 upper layer 在时间点 t_1 的 unit 数量。

这说明 expression OT 不是直接计算 lower unit 到 upper unit 的跨层转移。它是分别在 lower 层级内部做时间转移，在 upper 层级内部做时间转移。后续 EI/DI 或 vertical comparison 再比较 lower 与 upper 两个层级上的因果结构差异。

17. 两个方法的差异表

项目	clean_grn_cci_mix	clean_grn_cci_expr_mix
network builder	CleanGRNCCIMixBuilder	CleanGRNCCIEExpressionMixBuilder
继承关系	继承 LegacyMixedGRNCCIBuilder	继承 CleanGRNCCIMixBuilder
network_method	clean_grn_cci_mix	clean_grn_cci_expr_mix
intra GRN source	sample	sample_expression_weighted
intra GRN influence	$\tilde{w}_{g_a, g_b}$	$\tilde{w}_{g_a, g_b} \cdot a_u(g_a, g_b)$
inter CCI source	CCI-only	CCI-only
inter influence	$\tilde{s}_{u,v}^{(\ell, r)}$	$\tilde{s}_{u,v}^{(\ell, r)}$
是否把 GRN 混入 inter CCI	否	否
是否把表达连续权重乘进 intra GRN	否	是
默认表达活性模式	none	geometric_mean
特征空间	native_units	native_units
特征列	intra GRN gene pairs + inter CCI LR channels	intra GRN gene pairs + inter CCI LR channels
后续 expr_ot 输入	network feature, 不是 raw expression	network feature, 不是 raw expression

18. 对实验解释最重要的结论

第一, `clean_grn_cci_mix` 可以理解成一个“干净分离”的网络特征方案。它保留 intra GRN, 也保留 inter CCI, 但不再把 GRN 权重乘到 inter CCI 上。因此它适合回答: 如果细胞内部调控用 GRN, 细胞之间通讯只用 CCI, 那么这个网络表征是否足以产生更好的 PIJ 和 EI/DI 结果。

第二, `clean_grn_cci_expr_mix` 是在 `clean_grn_cci_mix` 基础上, 对 intra GRN 做了表达状态调制。它不是简单地把原始表达矩阵拼进 feature, 而是让每个 unit 内部的 GRN 边强度变成:

$$\text{GRN topology weight} \times \text{unit-specific expression activity}.$$

在这个公式里, GRN topology weight 对应 $\tilde{w}_{g_a, g_b}$, 也就是样本级 GRN 的归一化边权; unit-specific expression activity 对应 $a_u(g_a, g_b)$, 也就是 unit u 中 regulator-target 两端表达活性的联合函数。

第三, 两个方法的 CCI 部分完全一致, 都是:

inter feature = normalized COMMOT CCI score aggregated by outgoing LR channel.

在这个公式里, inter feature 表示 unit 的 inter CCI 特征; normalized COMMOT CCI score 对应 $\tilde{s}_{u,v}^{(\ell, r)}$; outgoing LR channel 表示按源 unit u 和 ligand-receptor channel (ℓ, r) 汇总, 而不是按目标 unit v 单独建列。

第四, `expr_ot` 里的“expression”在这里不是 raw expression, 而是 network feature distance。对于这两个 network method, `expr_ot` 实际做的是:

$$\begin{aligned} \text{GRN/CCI network feature} &\rightarrow \text{PCA} \rightarrow \text{MinMax scaling} \\ &\rightarrow \text{cosine cost} \rightarrow \text{Sinkhorn OT} \rightarrow P_{ij}. \end{aligned}$$

在这个流程里, P_{ij} 表示 source time 的第 i 个 unit 转移到 target time 的第 j 个 unit 的概率; i 表示 source unit index; j 表示 target unit index。

第五, 如果你看到 `clean_grn_cci_expr_mix` 比 `clean_grn_cci_mix` 结果更好, 一个合理解释是: 表达加权让 GRN 不再只是一个样本级静态拓扑, 而是变成了 unit-specific active GRN。也就是说, 同一条 GRN 边 $g_a \rightarrow g_b$ 在不同 unit u 里有不同强度:

$$I_{u, g_a, g_b}^{\text{intra, expr}} = \tilde{w}_{g_a, g_b} \cdot a_u(g_a, g_b).$$

在这个公式里, $I_{u, g_a, g_b}^{\text{intra, expr}}$ 表示 unit-specific 的表达加权 GRN 活性; $\tilde{w}_{g_a, g_b}$ 表示全样本 GRN 拓扑边权; $a_u(g_a, g_b)$ 表示 unit u 的表达活性。这个公式说明 `clean_grn_cci_expr_mix` 更像“样本级 GRN 结构 + unit 层面的表达激活状态”。

19. 可以在论文/报告里使用的表述

可以把 `clean_grn_cci_mix` 描述为:

We constructed a clean GRN-CCI mixed representation in which intra-unit regulatory activity is encoded by normalized sample-level GRN weights, while inter-unit communication activity is encoded solely by normalized COMMOT-derived ligand-receptor communication scores. This design avoids multiplying GRN weights into inter-cellular CCI edges and therefore separates intra-unit regulatory structure from inter-unit communication structure.

可以把 `clean_grn_cci_expr_mix` 描述为:

We further introduced an expression-weighted variant of the clean GRN-CCI representation. In this variant, each intra-unit GRN edge is weighted by both its normalized sample-level regulatory strength and the unit-specific expression activity of its regulator and target genes, whereas inter-unit CCI features remain CCI-only. This converts a static sample-level GRN topology into a unit-specific active regulatory feature representation.

中文可以写成:

`clean_grn_cci_mix` 将 unit 内部的 GRN 调控强度与 unit 之间的 CCI 通讯强度分开建模: intra 特征来自归一化 GRN 权重, inter 特征来自归一化 COMMOT CCI score, 不再在 inter 边上混入 GRN 权重。

`clean_grn_cci_expr_mix` 在 `clean_grn_cci_mix` 的基础上进一步引入 unit-specific gene expression activity, 对 intra GRN 边进行表达加权, 使同一条样本级 GRN 边在不同 unit 中具有不同的活跃程度; 但 inter CCI 部分仍然保持 CCI-only。